

EXPLORATORY DATA ANALYSIS

WITH

PANDAS

CHEAT SHEET

★ 2026 UPDATED EDITION — PANDAS 2.x READY ★

ANALYZE

VISUALIZE

TRANSFORM

CLEAN DATA

LOAD DATA

16 topics • 110+ commands • every outdated snippet fixed

Dream ✓ Plan ✓ Code ✓



THE EDA WORKFLOW

LOAD

INSPECT

CLEAN

TRANSFORM

ANALYZE

VISUALIZE

EXPORT

■ WHAT CHANGED IN MODERN PANDAS (2.x / 3.0) — READ THIS FIRST!

<code>df.append()</code>	removed in 2.0 → use <code>pd.concat([df1, df2])</code>
<code>df.applymap()</code>	removed in 2.1+ → use <code>df.map()</code>
<code>fillna(method='ffill')</code>	deprecated → use <code>df.ffill()</code> / <code>df.bfill()</code>
<code>resample('M')</code>	renamed in 2.2 → use 'ME' (also 'QE', 'YE')
<code>df.corr()</code> with text cols	now needs <code>corr(numeric_only=True)</code>
<code>pandas_profiling</code>	renamed → <code>ydata-profiling</code>
<code>inplace=True everywhere</code>	discouraged → assign back: <code>df = df.method()</code>
Copy-on-Write	default in pandas 3.0 — chained assignment like <code>df[a][b]=x</code> no longer works; use <code>df.loc[...]</code>

1. Data Loading & Import

Read CSV file	<code>df = pd.read_csv('file.csv')</code>
CSV with custom delimiter	<code>pd.read_csv('file.csv', sep=';')</code>
Read Excel file	<code>pd.read_excel('file.xlsx', sheet_name='Sheet1')</code>
Read JSON / XML / HTML	<code>pd.read_json('f.json');</code> <code>pd.read_xml('f.xml');</code> <code>pd.read_html(url)</code>
Read Parquet (fast + small) NEW	<code>pd.read_parquet('file.parquet')</code> <i>Preferred modern format for large data</i>
Read from SQL database	<code>from sqlalchemy import create_engine; engine = create_engine(url);</code> <code>pd.read_sql('SELECT * FROM t', engine)</code>
Read only needed columns NEW	<code>pd.read_csv('f.csv', usecols=['a', 'b'])</code> <i>Big memory saver</i>
Chunk very large files	<code>for chunk in pd.read_csv('big.csv', chunksize=100_000): process(chunk)</code>
PyArrow-backed dtypes NEW	<code>pd.read_csv('f.csv', dtype_backend='pyarrow', engine='pyarrow')</code> <i>pandas 2.x: faster reads, less memory, better NA handling</i>

2. Basic Data Inspection

Top / bottom rows	<code>df.head(); df.tail()</code>
Random sample of rows NEW	<code>df.sample(5)</code> <i>Less biased first look than head()</i>
Rows x columns / column names NEW	<code>df.shape; df.columns.tolist()</code>
Data types of each column	<code>df.dtypes</code>
Index, columns, nulls, memory	<code>df.info()</code>
Summary statistics (numeric)	<code>df.describe()</code>
Summary incl. text columns NEW	<code>df.describe(include='all')</code>
Memory usage (accurate)	<code>df.memory_usage(deep=True)</code>



3. Data Cleaning — Missing Values

Count missing per column	<code>df.isnull().sum()</code>
Percent missing per column NEW	<code>df.isnull().mean() * 100</code>
Fill missing with a value	<code>df.fillna(value) / df['col'].fillna(df['col'].median())</code>
Forward / backward fill UPDATED	<code>df['col'].ffill(); df['col'].bfill()</code> <i>fillna(method='ffill') is deprecated in pandas 2.x — use ffill()/bfill()</i>
Interpolate missing values	<code>df['col'].interpolate()</code>
Drop rows / columns with NA	<code>df.dropna(); df.dropna(axis=1); df.dropna(subset=['col'])</code>
Best automatic dtypes + pd.NA NEW	<code>df = df.convert_dtypes()</code> <i>Nullable Int64/boolean/string dtypes</i>

4. Data Cleaning — Structure & Duplicates

Rename columns	<code>df.rename(columns={'old': 'new'})</code>
Clean all column names at once NEW	<code>df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')</code>
Drop columns	<code>df.drop(columns=['col1', 'col2'])</code>
Find duplicate rows	<code>df.duplicated().sum()</code>
Remove duplicates	<code>df.drop_duplicates(subset=['col'], keep='first')</code>
Replace values NEW	<code>df['col'].replace({'M': 'Male', 'F': 'Female'})</code>
Compare two DataFrames NEW	<code>df1.compare(df2)</code> <i>Shows exactly what changed</i>

5. Indexing & Selection

Select one / many columns	<code>df['col']; df[['col1', 'col2']]</code>
Rows by position	<code>df.iloc[0:5]; df.iloc[0:5, 0:3]</code>
Rows by label	<code>df.loc[0:5, ['col1', 'col2']]</code>
Conditional selection	<code>df[df['col'] > value]</code>
Multiple conditions NEW	<code>df[(df['a'] > 5) & (df['b'] == 'x')]</code> <i>Use & ~ with parentheses</i>
Filter with isin	<code>df[df['col'].isin([v1, v2])]</code>
Range filter NEW	<code>df[df['col'].between(10, 20)]</code>
Query method (SQL-like)	<code>df.query('col1 > value and col2 == "x"')</code>
Top / bottom N by value NEW	<code>df.nlargest(5, 'col'); df.nsmallest(5, 'col')</code> <i>Faster than sort_values().head()</i>
Select columns by dtype NEW	<code>df.select_dtypes(include='number')</code>



6. Type Conversion & Formatting

Convert data type	<code>df['col'] = df['col'].astype('int64')</code>
Safe numeric conversion NEW	<code>pd.to_numeric(df['col'], errors='coerce')</code> <i>Bad values become NaN instead of crashing</i>
Datetime conversion UPDATED	<code>pd.to_datetime(df['col'], format='%Y-%m-%d', errors='coerce')</code> <i>Pass format for speed & safety</i>
String operations	<code>df['col'].str.lower(); df['col'].str.strip()</code>
To category (memory saver)	<code>df['col'] = df['col'].astype('category')</code>
Ordered categories UPDATED	<code>df['col'] = pd.Categorical(df['col'], categories=['low', 'mid', 'high'], ordered=True)</code> <i>cat.set_categories inplace pattern is gone</i>
Set / reset index	<code>df.set_index('col'); df.reset_index(drop=True)</code>

7. Data Transformation

Apply function to a column	<code>df['col'].apply(lambda x: x * 2)</code>
Element-wise on whole frame UPDATED	<code>df.map(lambda x: x + 1)</code> <i>applymap() was removed in pandas 2.1+ — use .map()</i>
Create columns fluently NEW	<code>df = df.assign(total=df['price'] * df['qty'])</code>
Row-wise custom function	<code>df.apply(lambda r: f(r['a'], r['b']), axis=1)</code>
Chain custom steps NEW	<code>df.pipe(clean).pipe(add_features)</code> <i>Readable pipelines</i>
Bin numbers into groups NEW	<code>pd.cut(df['age'], bins=[0, 18, 60, 120], labels=['child', 'adult', 'senior'])</code>
Equal-size quantile bins NEW	<code>pd.qcut(df['income'], q=4)</code>

8. Grouping & Aggregation

Group by + aggregate	<code>df.groupby('col').agg({'sales': 'sum'})</code>
Group by multiple columns UPDATED	<code>df.groupby(['c1', 'c2'], observed=True).mean(numeric_only=True)</code> <i>numeric_only=True now required with text cols; observed=True for category groupers</i>
Multiple functions at once	<code>df.groupby('col')['val'].agg(['mean', 'sum', 'count'])</code>
Named aggregation (clean cols) NEW	<code>df.groupby('city').agg(avg_price=('price', 'mean'), n=('price', 'count'))</code> <i>Modern, readable output columns</i>
Transform (keep original shape)	<code>df.groupby('col')['val'].transform(lambda x: x - x.mean())</code>
Pivot table	<code>df.pivot_table(index='c1', columns='c2', values='v', aggfunc='mean')</code>
Cross tabulation UPDATED	<code>pd.crosstab(df['c1'], df['c2'], normalize='index')</code> <i>normalize gives % instead of counts</i>

★★★★★

9. Merging & Combining

Inner join	<code>pd.merge(df1, df2, on='key', how='inner')</code>
Left / right join	<code>pd.merge(df1, df2, on='key', how='left')</code> # or <code>how='right'</code>
Outer join	<code>pd.merge(df1, df2, on='key', how='outer')</code>
Different key names NEW	<code>pd.merge(df1, df2, left_on='id', right_on='user_id')</code>
Validate the join NEW	<code>pd.merge(df1, df2, on='key', validate='one_to_many')</code> <i>Catches duplicate-key bugs early</i>
Stack rows (append is gone!) UPDATED	<code>pd.concat([df1, df2], ignore_index=True)</code> <i>df.append() was removed in pandas 2.0 — always use pd.concat</i>
Combine side-by-side	<code>pd.concat([df1, df2], axis=1)</code>

10. Reshaping Data

Wide to long (melt)	<code>df.melt(id_vars=['id'], var_name='metric', value_name='value')</code>
Long to wide (pivot)	<code>df.pivot(index='id', columns='year', values='sales')</code>
Stack / unstack	<code>df.stack(); df.unstack()</code>
wide_to_long helper	<code>pd.wide_to_long(df, ['sales'], i='id', j='year')</code>
Explode list column to rows NEW	<code>df.explode('tags')</code>
One-hot encoding NEW	<code>pd.get_dummies(df, columns=['city'], drop_first=True)</code> <i>Essential prep for ML</i>

11. Time Series

Set datetime index	<code>df = df.set_index(pd.to_datetime(df['date']))</code>
Resample monthly UPDATED	<code>df.resample('ME').mean()</code> <i>pandas 2.2+: 'M' -> 'ME' (month-end), 'Q' -> 'QE', 'Y' -> 'YE'</i>
Rolling window	<code>df['col'].rolling(window=7).mean()</code>
Lag / shift for comparison	<code>df['prev'] = df['col'].shift(1)</code>
Time-based grouping UPDATED	<code>df.groupby(pd.Grouper(key='date', freq='ME')).sum()</code>
Extract date parts NEW	<code>df['date'].dt.year; df['date'].dt.month_name(); df['date'].dt.dayofweek</code>
Percent change over time NEW	<code>df['col'].pct_change()</code>

12. Text Data Operations

Contains substring UPDATED	<code>df[df['col'].str.contains('text', case=False, na=False)]</code> <i>na=False avoids errors on missing values</i>
Split into columns	<code>df['col'].str.split(' ', expand=True)</code>
Regex extraction	<code>df['col'].str.extract(r'(\d+)')</code>
Regex replace NEW	<code>df['col'].str.replace(r'\s+', ' ', regex=True)</code>
Length / starts / ends NEW	<code>df['col'].str.len(); .str.startswith('A'); .str.endswith('z')</code>



13. Statistical Analysis

Correlation matrix UPDATED	<code>df.corr(numeric_only=True)</code> <i>numeric_only=True is required in pandas 2.x if text columns exist</i>
Covariance matrix UPDATED	<code>df.cov(numeric_only=True)</code>
Value counts	<code>df['col'].value_counts()</code>
Value counts as % NEW	<code>df['col'].value_counts(normalize=True) * 100</code>
Unique values / count	<code>df['col'].unique(); df['col'].nunique()</code>
Skewness & kurtosis (built-in) UPDATED	<code>df['col'].skew(); df['col'].kurt()</code> <i>No SciPy needed — pandas has these built in</i>
Min-max normalization	<code>(df['c'] - df['c'].min()) / (df['c'].max() - df['c'].min())</code>
Z-score standardization	<code>(df['c'] - df['c'].mean()) / df['c'].std()</code>
Quantiles / IQR outlier bounds NEW	<code>q1, q3 = df['c'].quantile([.25, .75]); iqr = q3 - q1</code> <i>Outliers: < q1-1.5*iqr or > q3+1.5*iqr</i>

14. Visualization (pandas + seaborn)

Histogram with bins	<code>df['col'].hist(bins=20)</code>
Boxplot / grouped boxplot	<code>df.boxplot(column='num_col', by='cat_col')</code>
Scatter / line / bar	<code>df.plot.scatter(x='a', y='b'); df.plot.line();</code> <code>df['c'].value_counts().plot.bar()</code>
Plot style	<code>import matplotlib.pyplot as plt; plt.style.use('ggplot'); df.plot();</code> <code>plt.show()</code>
Seaborn pairplot	<code>import seaborn as sns; sns.pairplot(df)</code>
Correlation heatmap UPDATED	<code>sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')</code>
Count plot for categories NEW	<code>sns.countplot(data=df, x='col')</code>
Visualize missing data NEW	<code>import missingno as msno; msno.matrix(df)</code> <i>pip install missingno</i>

15. Profiling & Data Quality

Automatic profile report UPDATED	<code>from ydata_profiling import ProfileReport;</code> <code>ProfileReport(df).to_file('report.html')</code> <i>pandas_profiling was renamed — install with: pip install ydata-profiling</i>
Quick alternative profiler NEW	<code>import sweetviz as sv; sv.analyze(df).show_html()</code>
Assert data validation	<code>assert df.notnull().all().all(), 'Missing values found!'</code>
Check expected dtypes NEW	<code>assert df['id'].dtype == 'int64'</code>



16. Performance, Big Data & Export

Reduce memory with category	<code>df['col'] = df['col'].astype('category')</code>
Efficient expressions	<code>df.eval('total = price * qty');</code> <code>df.query('price > 100')</code>
Sample for quick insights	<code>df.sample(n=1000, random_state=42)</code>
Dask for out-of-memory data UPDATED	<code>import dask.dataframe as dd; ddf = dd.read_csv('big-*.csv')</code> <i>Prefer dd.read_csv over from_pandas for big files</i>
Polars — the fast new option NEW	<code>import polars as pl; pl.read_csv('f.csv')</code> <i>Modern alternative, often 10x+ faster (swifter is outdated)</i>
Write CSV (no index column!) UPDATED	<code>df.to_csv('out.csv', index=False)</code> <i>index=False avoids the junk 'Unnamed: 0' column</i>
Write Excel / JSON / SQL	<code>df.to_excel('o.xlsx', index=False); df.to_json('o.json');</code> <code>df.to_sql('t', engine)</code>
Write Parquet (recommended) NEW	<code>df.to_parquet('out.parquet')</code> <i>Smaller + faster than CSV, keeps dtypes</i>

★ GOLDEN EDA TIPS

✓ Always start with `df.info()`, `df.describe()` and missing-value checks ✓ Never trust data before checking duplicates and dtypes ✓ Visualize before you conclude ✓ Prefer Parquet over CSV for saving work ✓ Use `random_state` for reproducibility ✓ Practice daily on real datasets (Kaggle!) — Consistency = Success